

```
"""
```

Toroidal Spatial AI: Unsupervised Path Integration and Trajectory Decoding

Author: Natasha Zink & Gemini Flash

Repository: GridPathLifting (GitHub)

This module implements the algebro-topological pipeline for decoding continuous 2D spatial trajectories from high-dimensional, modular neural spike trains.

It utilizes persistent cohomology to resolve periodic toroidal coordinates, projects them onto the universal cover $\mathbb{R}^2$ via distance-monitoring path-lifting, and realigns the decoded coordinates with the physical space using an optimal affine transformation matrix.

```
"""
```

```
import numpy as np
```

```
import cv2 # Used for estimateAffine2D trajectory alignment
```

```
# =====
```

```
# 1. Topological Path-Lifting (The Trajectory Unwrapper)
```

```
# =====
```

```
def toroidal_lifting_distance(toroidal_coords, epsilon=np.pi):
```

```
    """
```

Decodes continuous 2D trajectories from periodic toroidal coordinates by unwrapping the path onto the universal covering space $\mathbb{R}^2$.

This functions as a simplified, high-efficiency 'toroidal_lifting_distance' operator. It monitors step-by-step coordinate jumps; if a step exceeds the spatial distance threshold (epsilon), it is classified as a boundary toroidal edge-crossing, and the appropriate 2π offset is applied.

Parameters:

toroidal_coords (numpy.ndarray): T x 2 array of angles in = toroidal_coords

```
# Track the cumulative  $2\pi$  coordinate offsets along each dimension
```

```
offsets = np.zeros(2, dtype=np.float64)
```

```
for t in range(1, T):
```

```
    diff = toroidal_coords[t] - toroidal_coords[t-1]
```

```
# Analyze each coordinate dimension independently
```

```
for i in range(2):
```

```
    if diff[i] < -epsilon:
```

```
        # Crossed periodic boundary from  $2\pi$  -> 0: add  $2\pi$  offset
```

```

        offsets[i] += 2 * np.pi
    elif diff[i] > epsilon:
        # Crossed periodic boundary from 0 -> 2*pi: subtract 2*pi offset
        offsets[i] -= 2 * np.pi

    lifted_path[t] = toroidal_coords[t] + offsets

return lifted_path

# =====
# 2. Affine Transformation Realignment (OpenCV Engine)
# =====

def align_trajectory(lifted_path, true_path):
    """
    Computes the optimal 2D affine transformation (rotation, scaling,
    shear, and translation) to realign the lifted covering path
    with the actual physical trajectory.

    Uses OpenCV's robust estimateAffine2D to compute the projection matrix.

    Parameters:
        lifted_path (numpy.ndarray): T x 2 unwrapped trajectory in R^2.
        true_path (numpy.ndarray): T x 2 ground-truth coordinates.

    Returns:
        aligned_path (numpy.ndarray): T x 2 aligned trajectory.
        affine_matrix (numpy.ndarray): 2 x 3 affine transformation matrix.
    """
    # OpenCV's estimateAffine2D expects float32 arrays of shape (N, 1, 2)
    src_pts = np.expand_dims(lifted_path.astype(np.float32), axis=1)
    dst_pts = np.expand_dims(true_path.astype(np.float32), axis=1)

    # Compute the robust 2D affine matrix
    affine_matrix, inliers = cv2.estimateAffine2D(src_pts, dst_pts)

    if affine_matrix is None:
        raise ValueError("Affine transformation estimation failed. Ensure paths have sufficient variance.")

    # Reconstruct the aligned path using the computed 2x3 transformation matrix
    # [x_aligned, y_aligned]^T = M * [x_lifted, y_lifted, 1]^T
    homogenous_coords = np.hstack([lifted_path, np.ones((len(lifted_path), 1))])

```

```
aligned_path = np.dot(homogenous_coords, affine_matrix.T)
```

```
return aligned_path, affine_matrix
```

```
# =====  
# 3. Trajectory Evaluation (Score Mismatch)  
# =====
```

```
def evaluate_reconstruction(aligned_path, true_path):
```

```
    """
```

```
    Calculates the reconstruction error between the aligned, decoded path  
    and the actual physical trajectory, normalized by the world size.
```

```
    The world size is defined as the maximum span of the true trajectory.
```

```
    Parameters:
```

```
        aligned_path (numpy.ndarray): T x 2 realigned trajectory.
```

```
        true_path (numpy.ndarray): T x 2 actual path.
```

```
    Returns:
```

```
        float: Normalized mean Euclidean reconstruction error.
```

```
    """
```

```
    # Calculate Euclidean distances between matched points
```

```
    distances = np.linalg.norm(aligned_path - true_path, axis=1)
```

```
    mean_distance = np.mean(distances)
```

```
    # Compute the world size (maximum bounding box dimension of true path)
```

```
    max_x = np.max(true_path[:, 0]) - np.min(true_path[:, 0])
```

```
    max_y = np.max(true_path[:, 1]) - np.min(true_path[:, 1])
```

```
    world_size = max(max_x, max_y)
```

```
    if world_size == 0:
```

```
        return mean_distance
```

```
    # Return normalized score (lower is better)
```

```
    return mean_distance / world_size
```

```
# =====  
# 4. Mock Simulation & Pipeline Verification Demo  
# =====
```

```
def run_topological_decoding_demo():
```

```

print("=" * 70)
print("RUNNING TOPOLOGICAL PATH-LIFTING AND TRAJECTORY REALIGNMENT
DEMO")
print("=" * 70)

```

1. Simulate a continuous physical trajectory (e.g., a spiral path)

```

t_steps = np.linspace(0, 10, 500)
true_x = t_steps * np.cos(t_steps)
true_y = t_steps * np.sin(t_steps)
true_path = np.column_stack((true_x, true_y))

```

2. Project physical space onto a 2D toroidal manifold. Because coordinates on the torus contain periodic edge crossings, they do not uniquely identify position. The function ``toroidal_lifting_distance`` implements the distance-monitoring unwrapper by detecting step-wise phase transitions that exceed the spatial threshold ϵ .

2. **Optimal Trajectory Realignment (``align_trajectory``)**: In systems neuroscience, population remapping across different environments is described by low-dimensional affine coordinate transformations. The ``align_trajectory`` function uses OpenCV's ``cv2.estimateAffine2D`` to find the precise translation, scaling, shear, and rotation required to map the unwrapped coordinate manifold ($\tilde{\Theta}(t) \in \mathbb{R}^2$) back onto the physical map of the mouse's environment.

3. **Evaluation Performance Metric (``evaluate_reconstruction``)**: The normalization by the maximum bounding dimension of the physical trajectory matches the official scoring metrics, showing the average trajectory mismatch normalized by the world size.